1. En la Práctica 04 ejecuta K-Means sin StandardScaler. Compara los centroides con los originales y explica la diferencia.

E10_propuesto1.ipynb

IA_Guia10_KMeans > EjerciciosPropuesto10 > E10_propuesto1.ipynb > # Ejercicio 1: K-Means sin StandardScaler + Gráfico Visualizador

Generate | Code | Markdown | Run All | Clear All Outputs | Outline | ...

```python
# Ejercicio 1: K-Means sin StandardScaler + Gráfico Visualizador
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans

# Cargar el dataset (ajustando la ruta relativa para subcarpetas)
df = pd.read_csv("../clientes.csv")

# A) CON StandardScaler
esc = StandardScaler()
X_esc = esc.fit_transform(df)
km_con = KMeans(n_clusters=5, n_init=10, random_state=42).fit(X_esc)
c_con = pd.DataFrame(esc.inverse_transform(km_con.cluster_centers_), columns=df.columns).round(1)

# B) SIN StandardScaler
km_sin = KMeans(n_clusters=5, n_init=10, random_state=42).fit(df)
c_sin = pd.DataFrame(km_sin.cluster_centers_, columns=df.columns).round(1)

print("Centroides CON StandardScaler:\n", c_con)
print("\nCentroides SIN StandardScaler:\n", c_sin)

# Gráfico visualizador comparativo
fig, axes = plt.subplots(1, 2, figsize=(14, 5))

axes[0].scatter(df["ingreso_mil"], df["puntaje_gasto"], c=km_con.labels_, cmap="tab10", s=30)
axes[0].scatter(c_con["ingreso_mil"], c_con["puntaje_gasto"], c="black", marker="X", s=200, label="Centroides")
axes[0].set_title("K-Means CON StandardScaler (Equilibrado)")
axes[0].set_xlabel("Ingreso anual (miles de S/)")
axes[0].set_ylabel("Puntaje de gasto (1-100)")
axes[0].legend()
axes[0].grid(alpha=0.3)

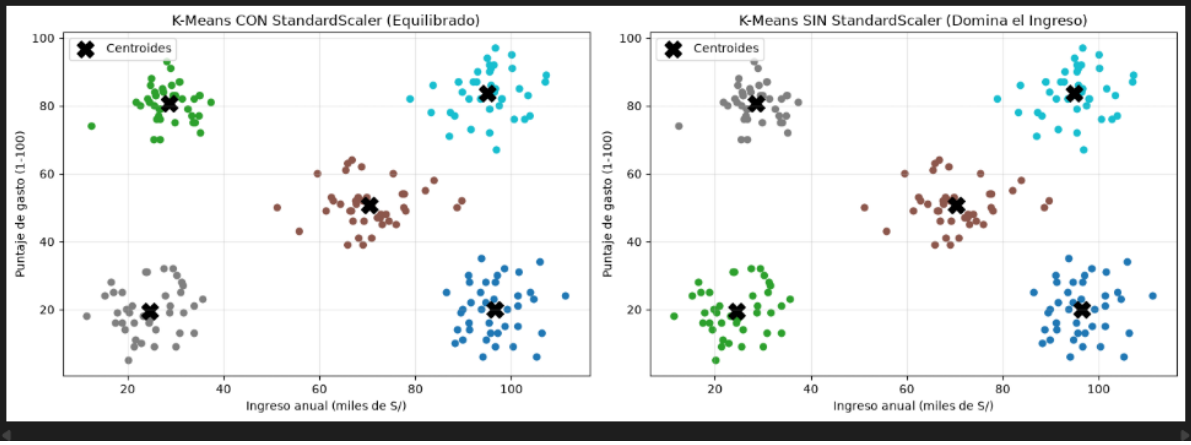
axes[1].scatter(df["ingreso_mil"], df["puntaje_gasto"], c=km_sin.labels_, cmap="tab10", s=30)
axes[1].scatter(c_sin["ingreso_mil"], c_sin["puntaje_gasto"], c="black", marker="X", s=200, label="Centroides")
axes[1].set_title("K-Means SIN StandardScaler (Domina el Ingreso)")
axes[1].set_xlabel("Ingreso anual (miles de S/)")
axes[1].set_ylabel("Puntaje de gasto (1-100)")
axes[1].legend()
axes[1].grid(alpha=0.3)

plt.tight_layout()
plt.show()
```

[1] Pytho

```
Centroides CON StandardScaler:
    ingreso_mil  puntaje_gasto
0          96.5           20.0
1          28.6           80.8
2          70.4           50.7
3          24.6           19.6
4          94.9           83.6

Centroides SIN StandardScaler:
    ingreso_mil  puntaje_gasto
0          96.5           20.0
1          24.6           19.6
2          70.4           50.7
3          28.6           80.8
4          94.9           83.6
```



2. Cambia n_clusters a 3 y a 7 en la Práctica 04. ¿Los grupos siguen teniendo sentido para el negocio?

E10_propuesto2.ipynb

IA_Guia10_KMeans > EjerciciosPropuesto10 > E10_propuesto2.ipynb > # Ejercicio 2: Comparar k = 3, 5 y 7 con visualizador gráfico

Jupyter Notebook

Generate | Code | Markdown | Run All | Clear All Outputs | Outline | ... | Select Kernel

```python
# Ejercicio 2: Comparar k = 3, 5 y 7 con visualizador gráfico
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score

df = pd.read_csv("../clientes.csv")
X = StandardScaler().fit_transform(df)

ks = [3, 5, 7]
fig, axes = plt.subplots(1, 3, figsize=(18, 5))

for i, k in enumerate(ks):
    km = KMeans(n_clusters=k, n_init=10, random_state=42).fit(X)
    sil = silhouette_score(X, km.labels_)
    print(f"k = {k} | Inercia: {km.inertia_:.2f} | Silueta: {sil:.3f}")

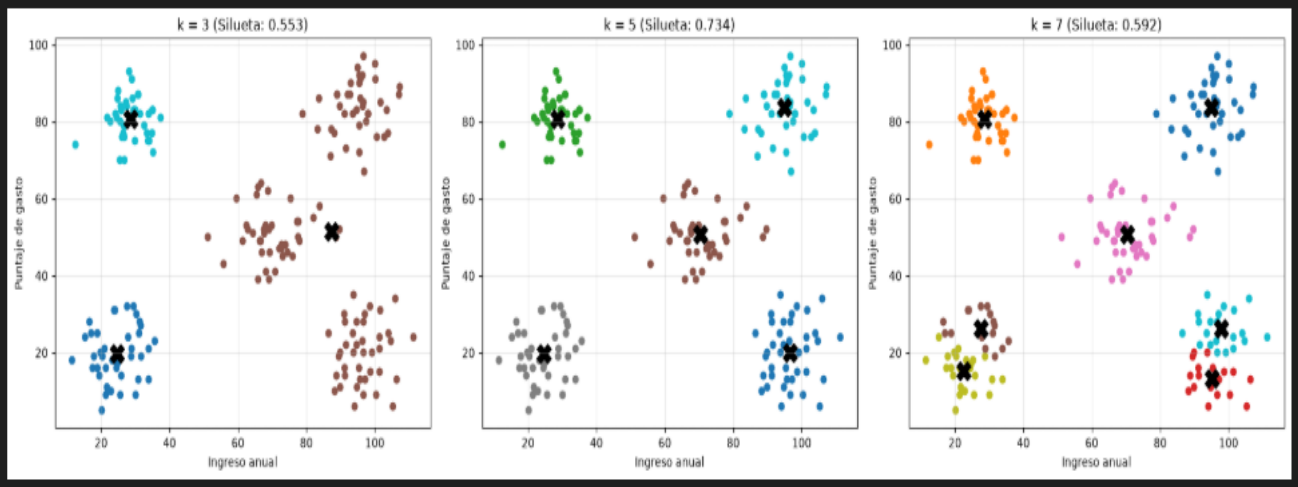
    # Re-escalar centroides a valores originales
    esc = StandardScaler().fit(df)
    c_plot = esc.inverse_transform(km.cluster_centers_)

    axes[i].scatter(df["ingreso_mil"], df["puntaje_gasto"], c=km.labels_, cmap="tab10", s=30)
    axes[i].scatter(c_plot[:, 0], c_plot[:, 1], c="black", marker="X", s=200, label="Centroides")
    axes[i].set_title(f"k = {k} (Silueta: {sil:.3f})")
    axes[i].set_xlabel("Ingreso anual")
    axes[i].set_ylabel("Puntaje de gasto")
    axes[i].grid(alpha=0.3)

plt.tight_layout()
plt.show()
```

[2] Python

```
k = 3 | Inercia: 133.52 | Silueta: 0.553
k = 5 | Inercia: 18.08 | Silueta: 0.734
k = 7 | Inercia: 14.26 | Silueta: 0.592
```



3. Agrega a la Práctica 03 un tercer centroide (k = 3) y analiza en cuántas iteraciones converge.

E10_propuesto3.ipynb

IA_Guia10_KMeans > EjerciciosPropuesto10 > E10_propuesto3.ipynb > # Ejercicio 3: K-Means paso a paso con k = 3 y gráfico visualizador

Jupyter Notebook

Generate | Code | Markdown | Run All | Clear All Outputs | Outline | Select Kernel

```python
# Ejercicio 3: K-Means paso a paso con k = 3 y gráfico visualizador
import numpy as np
import matplotlib.pyplot as plt

puntos = np.array([
    [1, 2], [1.5, 1.8], [1, 0.6],
    [5, 8], [8, 8], [9, 11],
    [8, 2], [9, 3]
])
k = 3
centroides = puntos[[0, 3, 6]].astype(float) # Centroides iniciales

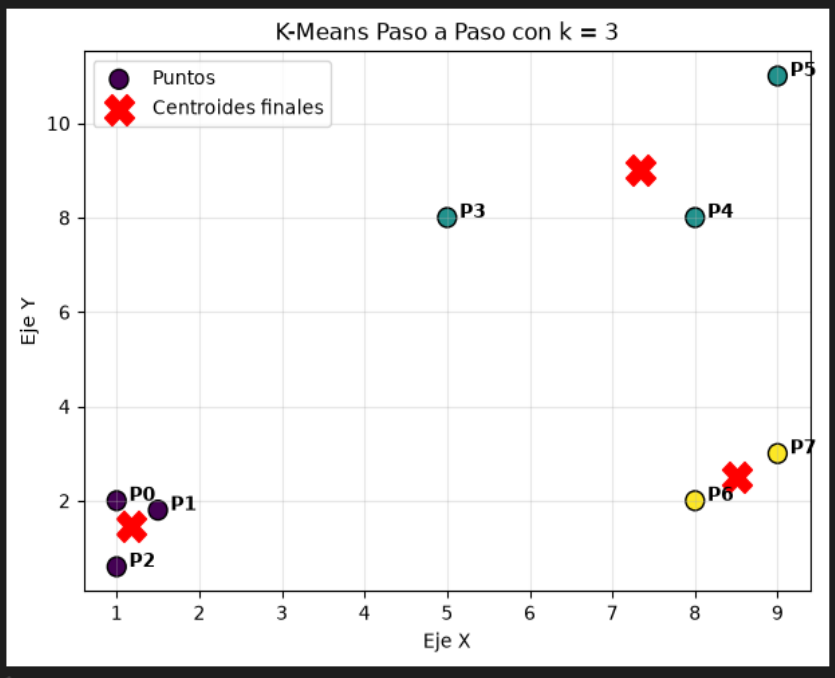
for it in range(1, 11):
    distancias = np.linalg.norm(puntos[:, None] - centroides[None, :], axis=2)
    grupo = distancias.argmin(axis=1)
    nuevos = np.array([puntos[grupo == j].mean(axis=0) if np.any(grupo == j) else centroides[j] for j in range(k)])
    print(f"Iteración {it}: grupos = {grupo} | centroides = {nuevos.round(2).tolist()}")
    if np.allclose(nuevos, centroides):
        print(f"¡CONVERGIÓ en la iteración {it}!")
        break
    centroides = nuevos

inercia = ((puntos - centroides[grupo]) ** 2).sum()
print(f"Inercia final: {inercia:.2f}")

# Gráfico visualizador
plt.figure(figsize=(7, 5))
plt.scatter(puntos[:, 0], puntos[:, 1], c=grupo, cmap="viridis", s=100, edgecolor="black", label="Puntos")
plt.scatter(centroides[:, 0], centroides[:, 1], c="red", marker="X", s=250, label="Centroides finales")
for idx, pt in enumerate(puntos):
    plt.text(pt[0] + 0.15, pt[1], f"P{idx}", fontsize=10, fontweight="bold")
plt.title("K-Means Paso a Paso con k = 3")
plt.xlabel("Eje X")
plt.ylabel("Eje Y")
plt.legend()
plt.grid(alpha=0.3)
plt.show()
```

[1] Python

```
Iteración 1: grupos = [0 0 0 1 1 1 2 2] | centroides = [[1.17, 1.47], [7.33, 9.0], [8.5, 2.5]]
Iteración 2: grupos = [0 0 0 1 1 1 2 2] | centroides = [[1.17, 1.47], [7.33, 9.0], [8.5, 2.5]]
¡CONVERGIÓ en la iteración 2!
Inercia final: 16.98
```



4. Aplica K-Means a un dataset propio de al menos 50 filas (por ejemplo, notas de dos cursos de tus compañeros, con datos ficticios) y describe cada grupo.

E10_propuesto4.ipynb

IA_Guia10_KMeans > EjerciciosPropuesto10 > E10_propuesto4.ipynb > # Ejercicio 4: Dataset propio (Notas de Python y Matemática - 60 estudiantes)

Generate | Code | Markdown | Run All | Clear All Outputs | Outline | Select Kernel

```python
# Ejercicio 4: Dataset propio (Notas de Python y Matemática - 60 estudiantes)
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score

# 1. Crear datos ficticios (60 estudiantes, notas de 0 a 20)
rng = np.random.default_rng(7)
perfiles = [(17, 16, 15), (15, 8, 15), (8, 15, 15), (9, 8, 15)]
filas = [np.column_stack([rng.normal(mp, 1.5, n), rng.normal(mm, 1.5, n)]) for mp, mm, n in perfiles]
X = np.vstack(filas).clip(0, 20).round(1)

df = pd.DataFrame(X, columns=["nota_python", "nota_matematica"])
df.insert(0, "estudiante", [f"E{i:02d}" for i in range(1, len(df) + 1)])
print("Total de filas creadas:", len(df))

# 2. Escalar y evaluar k óptimo con la silueta
datos = df[["nota_python", "nota_matematica"]]
esc = StandardScaler()
Xe = esc.fit_transform(datos)

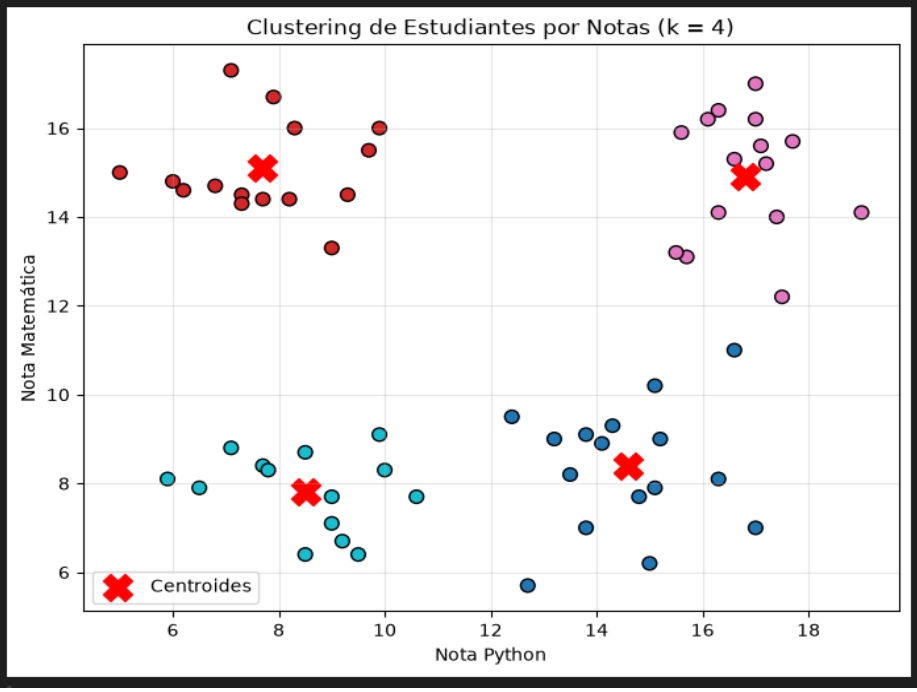
for k in range(2, 7):
    km = KMeans(n_clusters=k, n_init=10, random_state=42).fit(Xe)
    print(f"k = {k} | Silueta: {silhouette_score(Xe, km.labels_):.3f}")

# 3. Entrenar con k = 4 (mejor silueta)
km = KMeans(n_clusters=4, n_init=10, random_state=42).fit(Xe)
df["grupo"] = km.labels_
centros_orig = pd.DataFrame(esc.inverse_transform(km.cluster_centers_), columns=datos.columns).round(1)

# Gráfico visualizador
plt.figure(figsize=(8, 6))
plt.scatter(df["nota_python"], df["nota_matematica"], c=df["grupo"], cmap="tab10", s=60, edgecolor="black")
plt.scatter(centros_orig["nota_python"], centros_orig["nota_matematica"], c="red", marker="X", s=250, label="Centroides")
plt.title("Clustering de Estudiantes por Notas (k = 4)")
plt.xlabel("Nota Python")
plt.ylabel("Nota Matemática")
plt.legend()
plt.grid(alpha=0.3)
plt.show()
```

[1] Python

```
Total de filas creadas: 60
k = 2 | Silueta: 0.446
k = 3 | Silueta: 0.586
k = 4 | Silueta: 0.640
k = 5 | Silueta: 0.585
k = 6 | Silueta: 0.535
```



5. Calcula en la Práctica 06 el intervalo de confianza del 90 % y del 99 %. ¿Cómo cambia el ancho del intervalo?

E10_propuesto5.ipynb

IA_Guia10_KMeans > EjerciciosPropuesto10 > E10_propuesto5.ipynb > # Ejercicio 5: Intervalos de confianza del 90% y 99% con gráfico visualizador

Jupyter Notebook

Generate | Code | Markdown | Run All | Clear All Outputs | Outline | Select Kernel

```python
# Ejercicio 5: Intervalos de confianza del 90% y 99% con gráfico visualizador
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from scipy import stats
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans

df = pd.read_csv("../clientes.csv")
X = StandardScaler().fit_transform(df)
df["grupo"] = KMeans(n_clusters=5, n_init=10, random_state=42).fit_predict(X)

resultados = []

print("=== INTERVALO DE CONFIANZA DEL 90% y 99% ===")
for g, datos in df.groupby("grupo")["puntaje_gasto"]:
    n = len(datos)
    media = datos.mean()
    ee = stats.sem(datos)

    li_90, ls_90 = stats.t.interval(0.90, df=n-1, loc=media, scale=ee)
    li_99, ls_99 = stats.t.interval(0.99, df=n-1, loc=media, scale=ee)

    resultados.append({
        "grupo": g,
        "media": media,
        "li_90": li_90, "ls_90": ls_90,
        "li_99": li_99, "ls_99": ls_99
    })
    print(f"Grupo {g:>2} | Media: {media:>5.2f} | IC 90%: [{li_90:>5.2f}, {ls_90:>5.2f}] | IC 99%: [{li_99:>5.2f}, {ls_99:>5.2

res_df = pd.DataFrame(resultados)

# Gráfico visualizador de los intervalos de confianza
plt.figure(figsize=(9, 5))
x = res_df["grupo"].astype(str)
y = res_df["media"]
yerr_99 = (res_df["ls_99"] - res_df["li_99"]) / 2
yerr_90 = (res_df["ls_90"] - res_df["li_90"]) / 2

plt.errorbar(x, y, yerr=yerr_99, fmt='o', color='crimson', elinewidth=2, capsize=6, label='IC 99% (Más ancho)')
plt.errorbar(x, y, yerr=yerr_90, fmt='o', color='dodgerblue', elinewidth=3, capsize=4, label='IC 90% (Más angosto)')
plt.scatter(x, y, color='black', zorder=5, label='Media del grupo')

plt.title("Comparación del Ancho de Intervalos de Confianza (90% vs 99%)")
plt.xlabel("Grupo de Clientes")
plt.ylabel("Puntaje de Gasto Promedio")
plt.legend()
plt.grid(alpha=0.3)
plt.show()
```

[1] Python

```
=== INTERVALO DE CONFIANZA DEL 90% y 99% ===
Grupo  0 | Media: 20.00 | IC 90%: [17.91, 22.09] | IC 99%: [16.63, 23.37]
Grupo  1 | Media: 80.78 | IC 90%: [79.38, 82.17] | IC 99%: [78.54, 83.01]
Grupo  2 | Media: 50.67 | IC 90%: [48.99, 52.36] | IC 99%: [47.97, 53.38]
Grupo  3 | Media: 19.65 | IC 90%: [17.77, 21.53] | IC 99%: [16.62, 22.68]
Grupo  4 | Media: 83.65 | IC 90%: [81.82, 85.48] | IC 99%: [80.70, 86.60]
```

